

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

SİSTEM PROGRAMLAMA
2025-2026 BAHAR DÖNEMİ PROJESİ

tarsau - Arşivleme Programı

Okan Koca - B231210016
Yasin Dalgın - B221210016

GitHub: <https://github.com/OkanKoca/system-programming-tarsau>

Mayıs 2026

1. Giriş

Bu projede, tar, rar veya zip benzeri çalışan ancak sıkıştırma yapmayan "tarsau" adlı bir arşivleme programı geliştirilmiştir. Program, Linux/Unix işletim sistemi üzerinde C dilinde yazılmış olup komut satırından çalıştırılmaktadır.

Program iki temel modda çalışmaktadır: birden fazla ASCII metin dosyasını tek bir .sau arşiv dosyasına birleştirme (-b modu) ve bir .sau arşiv dosyasından dosyaları geri çıkarma (-a modu). Arşivleme sırasında dosya izinleri (permissions) korunmakta ve çıkarma sırasında geri yüklenmektedir.

2. Tasarım ve Mimari

2.1 Genel Yapı

Program tek bir C kaynak dosyasından (src/tarsau.c) oluşmaktadır. Proje yapısı aşağıdaki gibidir:

```
system_programming_project/
├── src/
│   ├── tarsau.c      # Ana program
│   └── Makefile      # Derleme kuralları
├── README.md
```

2.2 SAU Arşiv Dosya Formatı

Arşiv dosyası iki bölümden oluşmaktadır: organizasyon (header) bölümü ve veri bölümü.

Bölüm	Açıklama
Header (10 bayt)	Organizasyon bölümünün boyutunu ASCII sayısal formatında içerir (örn: 0000000033)
Organizasyon	Her dosya için dosya_adı,izinler,boyut formatında kayıtlar. Alanlar virgülle, kayıtlar ile ayrılır.
Veri	Dosya içerikleri herhangi bir ayırıcı olmadan art arda ASCII olarak yerleştirilir.

Örnek arşiv dosyası yapısı:

```
0000000033|t1,644,29||t2,755,28||t3,600,21|Bu birinci dosyanin icerigi.
Bu ikinci dosyanin icerigi.
Ucuncu dosya burada.
```

2.3 Program Akışı

Program çalıştırıldığında önce komut satırı argümanları ayrıştırılır. -b parametresi arşivleme modunu, -a parametresi çıkarma modunu aktive eder. Her iki mod da gerekli doğrulama adımlarını gerçekleştirir ve hatalı durumlarda anlamlı hata mesajları üretir.

3. Fonksiyonlar ve Kod Açıklamaları

3.1 Sabitler ve Makrolar

```
#define MAX_FILES 32
#define MAX_TOTAL_SIZE (200UL * 1024 * 1024)
#define HEADER_SIZE 10
#define DEFAULT_OUTPUT "a.sau"
```

MAX_FILES: Aynı anda arşivlenebilecek maksimum dosya sayısını, MAX_TOTAL_SIZE: toplam dosya boyutu sınırını (200 MB), HEADER_SIZE: arşiv dosyasının başlık bölümü uzunluğunu ve DEFAULT_OUTPUT: varsayılan çıktı dosya adını belirler.

3.2 is_text_file() - Metin Dosyası Kontrolü

```
static int is_text_file(const char *path) {
    FILE *f = fopen(path, "rb");
    if (!f) return 0;
    int ch;
    while ((ch = fgetc(f)) != EOF) {
        if (ch == '\\0') { fclose(f); return 0; }
        unsigned char uc = (unsigned char)ch;
        if (uc > 127) { fclose(f); return 0; }
    }
    fclose(f);
    return 1;
}
```

Bu fonksiyon dosyayı binary modda açarak her baytı kontrol eder. NUL karakter (0x00) veya 127'den büyük değere sahip bayt bulursa dosya ASCII metin dosyası değildir ve 0 döner. Tüm baytlar geçerliyse 1 döner.

3.3 get_file_size() ve get_permissions()

```
static long get_file_size(const char *path) {
    struct stat st;
    if (stat(path, &st) != 0) return -1;
    return (long)st.st_size;
}

static mode_t get_permissions(const char *path) {
    struct stat st;
    if (stat(path, &st) != 0) return 0;
    return st.st_mode & 0777;
}
```

stat() sistem çağrısı kullanılarak dosya boyutu ve erişim izinleri elde edilir. İzinler 0777 maskesi ile filtrelenerek sadece kullanıcı/grup/diğer okuma-yazma-çalıştırma bitleri alınır.

3.4 archive_files() - Arşivleme Fonksiyonu

```
static int archive_files(int file_count, char *files[],
                        const char *output) {
    // 1. Doğrulama: dosya sayısı, boyut, format
    // 2. Organizasyon bolumu olusturma
    // 3. Header + organizasyon + veri yazma
}
```

Arşivleme fonksiyonu üç aşamada çalışır:

1. Doğrulama: Dosya sayısı (maks. 32), toplam boyut (maks. 200 MB) ve her dosyanın ASCII metin formatı kontrol edilir.

2. Organizasyon bölümü: Her dosya için "|dosya_adı,izinler,boyut|" formatında kayıt oluşturulur.
3. Önce 10 baytlık header, ardından organizasyon bölümü ve son olarak tüm dosya içerikleri art arda yazılır.

Organizasyon bölümünün oluşturulması:

```
char org_section[65536];
int org_len = 0;
for (int i = 0; i < file_count; i++) {
    int written = snprintf(org_section + org_len,
        sizeof(org_section) - org_len,
        "|%s,%o,%ld|", files[i], perms[i], sizes[i]);
    org_len += written;
}
```

3.5 extract_files() - Çıkarma Fonksiyonu

```
static int extract_files(const char *archive,
    const char *dest_dir) {
    // 1. Header okuma (ilk 10 bayt)
    // 2. Organizasyon bolumu parse etme
    // 3. Hedef dizin olusturma
    // 4. Dosyalari cikarma + chmod ile izin atama
}
```

Çıkarma fonksiyonunun temel adımları:

4. Arşiv dosyasının ilk 10 baytı okunarak organizasyon bölümünün boyutu elde edilir.
5. Organizasyon bölümü | ve , karakterleri ile parse edilerek dosya adları, izinler ve boyutlar çıkarılır.
6. Hedef dizin belirtilmişse ve mevcut değilse mkdir() ile oluşturulur.
7. Her dosya, boyutu kadar veri okunarak hedef dizine yazılır ve chmod() ile orijinal izinleri geri yüklenir.

Organizasyon bölümü parse etme:

```
char *p = org_section;
while (*p && file_count < MAX_FILES) {
    if (*p == '|') p++;
    char *end = strchr(p, '|');
    // record: "dosya_adı,izinler,boyut"
    char *comma1 = strchr(record, ',');
    char *comma2 = strchr(comma1 + 1, ',');
    // Parse: dosya adı, izinler (octal), boyut
}
```

3.6 main() - Argüman Ayırıştırma

```
int main(int argc, char *argv[]) {
    if (argc < 2) { print_usage(); return 1; }
    if (strcmp(argv[1], "-b") == 0) {
        // -b modu: dosya listesi + -o cikti
        return archive_files(file_count, files, output);
    } else if (strcmp(argv[1], "-a") == 0) {
        // -a modu: arşiv dosyası + dizin
        return extract_files(archive, dest_dir);
    }
}
```

main() fonksiyonu argv üzerinden -b veya -a modunu tespit eder. -b modunda dosya listesi ve opsiyonel -o parametresi, -a modunda arşiv dosyası ve opsiyonel hedef dizin ayrıştırılır.

4. Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c99
TARGET = tarsau
SRC = src/tarsau.c

all: $(TARGET)

$(TARGET): $(SRC)
    $(CC) $(CFLAGS) -o $(TARGET) $(SRC)

clean:
    rm -f $(TARGET)

.PHONY: all clean
```

Makefile, gcc derleyicisi ile -Wall ve -Wextra uyarı flag'leri ve C99 standardı kullanılarak derleme yapar. make komutu ile tarsau binary dosyası, make clean komutu ile temizlik yapılır.

5. Sınırlamalar ve Kurallar

Kural	Değer
Maksimum dosya sayısı	32
Maksimum toplam boyut	200 MB
Desteklenen dosya tipleri	Yalnızca ASCII metin dosyaları
Varsayılan çıktı dosyası	a.sau
İzin koruma	Dosya izinleri (rwx) arşivleme sırasında kaydedilir, çıkarmada geri yüklenir

6. Hata Yönetimi

Program, kullanıcı hatalarına karşı kapsamlı kontroller içermektedir:

Hata Durumu	Mesaj
Binary/uyumsuz dosya	<dosya> giris dosyasinin formati uyumsuzdur!
Geçersiz/bozuk arşiv	Arsiv dosyasi uygunsuz veya bozuk!
Dosya sayısı aşımı	En fazla 32 dosya arsilenebilir.
Boyut aşımı	Toplam dosya boyutu 200 MB sinirini asiyor.
Dosya açilamadi	<dosya> dosyasi acilamadi.

Tüm hata durumlarında program anlamlı bir mesaj yazdırarak düzgün şekilde sonlanır (graceful exit). Açılan dosyalar kapatılır ve ayrılan bellek serbest bırakılır.

7. Test ve Ekran Çıktıları

```
$ ./tarsau [cite: 186]
Kullanım: [cite: 187]
  tarsau -b dosya1 dosya2 ... [-o arxiv.sau] [cite: 188]
  tarsau -a arxiv.sau [dizin] [cite: 189]
```

```
$ echo "Bu birinci dosyanin icerigi." > t1
$ echo "Bu ikinci dosyanin icerigi." > t2
$ echo "Ucuncu dosya burada." > t3
$ echo "Dorduncu dosya .txt uzantili." > t4.txt
$ echo "Besinci dosya .dat uzantili." > t5.dat
$ ./tarsau -b t1 t2 t3 t4.txt t5.dat -o s1.sau
Dosyalar birlestirildi.
$ ./tarsau -a s1.sau d1
d1 dizininde t1, t2, t3, t4.txt, t5.dat dosyalari acildi.
```

7.1 Derleme

```
$ make
gcc -Wall -Wextra -std=c99 -o tarsau src/tarsau.c
```

Program hiçbir hata veya uyarı vermeden başarıyla derlenmektedir.

7.2 Arşivleme Testi (-b)

```
$ echo "Bu birinci dosyanin icerigi." > t1
$ echo "Bu ikinci dosyanin icerigi." > t2
$ echo "Ucuncu dosya burada." > t3
$ echo "Dorduncu dosya .txt uzantili." > t4.txt
$ echo "Besinci dosya .dat uzantili." > t5.dat
$ chmod 644 t1 && chmod 755 t2 && chmod 600 t3
```

```
$ ./tarsau -b t1 t2 t3 t4.txt t5.dat -o s1.sau
Dosyalar birlestirildi.
```

7.3 Çıkarma Testi (-a)

```
$ ./tarsau -a s1.sau d1
d1 dizininde t1, t2, t3, t4.txt, t5.dat dosyalari acildi.
```

7.4 Doğrulama (diff)

```
$ diff t1 d1/t1 && echo AYNI
AYNI
$ diff t2 d1/t2 && echo AYNI
AYNI
$ diff t3 d1/t3 && echo AYNI
AYNI
$ diff t4.txt d1/t4.txt && echo AYNI
AYNI
$ diff t5.dat d1/t5.dat && echo AYNI
AYNI
```

Tüm çıkarılan dosyalar orijinal dosyalarla birebir aynıdır.

7.5 Hata Senaryoları

Binary dosya reddi:

```
$ printf "\x00\x01\x02\xff" > binary_file
$ ./tarsau -b binary_file -o fail.sau
binary_file giris dosyasinin formati uyumsuzdur!
```

Bozuk arşiv dosyası:

```
$ echo "bozukdata" > bozuk.sau
$ ./tarsau -a bozuk.sau d2
Arşiv dosyasi uygunsuz veya bozuk!
```

Parametresiz kullanım:

```
$ ./tarsau
Kullanım:
  tarsau -b dosya1 dosya2 ... [-o arşiv.sau]
  tarsau -a arşiv.sau [dizin]
```

7.6 Varsayılan Çıktı Dosyası Testi

```
$ ./tarsau -b t1 t2
Dosyalar birlestirildi.
$ ls a.sau
a.sau
```

-o parametresi belirtilmediğinde varsayılan olarak a.sau dosyası oluşturulmaktadır.

7.7 Arşiv Dosyası İçerik Örneği

```
$ ./tarsau -b t1 t2 t3 -o mini.sau
Dosyalar birlestirildi.

$ head -c 100 mini.sau
0000000033|t1,644,29||t2,755,28||t3,600,21|Bu birinci dosyanin
icerigi.
Bu ikinci dosyanin icerigi.
Ucuncu...
```

Arşiv dosyasının ilk 10 baytı (0000000033) organizasyon bölümünün 33 bayt olduğunu gösterir. Ardından dosya kayıtları ve dosya içerikleri art arda gelmektedir.

8. Kullanılan Teknolojiler

Teknoloji	Kullanım Amacı
C	Program geliştirme dili
GCC	C derleyicisi
Make	Derleme otomasyonu
POSIX API	stat(), chmod(), mkdir() sistem çağrıları
Linux/WSL	Geliştirme ve test ortamı
GitHub	Versiyon kontrolü

9. Sonuç

Bu projede, "tarsau" adlı sıkıştırmasız bir arşivleme programı başarıyla geliştirilmiştir. Program, birden fazla ASCII metin dosyasını tek bir .sau arşiv dosyasına birleştirme ve bu arşiv dosyasından dosyaları geri çıkarma işlevlerini yerine getirmektedir.

Program, dosya izinlerinin korunması, binary dosya tespiti ve reddi, dosya sayısı ve boyut sınırlamalarının kontrolü, bozuk arşiv tespiti gibi tüm gereksinimleri karşılamaktadır. Tüm hata durumları için anlamlı mesajlar üretilmekte ve program her durumda düzgün şekilde sonlanmaktadır.